

Module 05 — VXLAN + EVPN Overlay

FOR DUMMIES

Goal: stretch a Layer-2 segment between leaf1 and leaf2 across the routed fabric using VXLAN, with BGP EVPN as the control plane — then, as the capstone, automate it with the module 04 model.

This is the payoff module. Everything before it was substrate: the fabric exists (03) and configures itself (04) *so that* this can run on top.

1. The problem, in one paragraph

Your underlay is beautiful and entirely Layer 3 — which means a VLAN on leaf1 cannot exist on leaf2. But workloads (VM migration, clustering, legacy apps) keep demanding L2 adjacency between endpoints in different racks. The old answer was stretching VLANs with trunks and spanning-tree across the fabric, which un-solves everything module 03 solved. The modern answer: keep the fabric routed, and tunnel L2 frames inside UDP packets between leaves. That's VXLAN. Each leaf gets a VTEP (tunnel endpoint — its loopback), and a VNI (VXLAN Network Identifier — a VLAN number with 24 bits of room instead of 12).

VXLAN alone still needs to learn which MACs live behind which VTEP — flood-and-learn works but scales badly. BGP EVPN replaces it: an address family where leaves *advertise* their MACs/hosts as BGP routes. The spines don't tunnel anything; they're *route reflectors*, passing EVPN routes between leaves so the leaves don't need a full mesh.

Underlay (OSPF) moves packets between loopbacks. Overlay (EVPN over iBGP) moves *reachability information*. VXLAN moves the actual frames. Three jobs, three layers, deliberately separate.

2. Give the leaves something to bridge

Real deployments bridge servers; we'll fake two with each leaf's Ethernet3 in VLAN 100 — actually, simpler and just as instructive: we'll verify with the leaves' own SVIs. On leaf1:

```
configure terminal
vlan 100
  name overlay-test
interface Vlan100
  ip address 172.16.100.1/24
  no autostate
```

On leaf2, the same with 172.16.100.2/24. Right now, these two can't ping each other — same subnet, no L2 path. That ping working is this module's finish line.

3. EVPN control plane (all four nodes)

Spines — route reflectors. On spine1 (spine2: swap in 10.255.0.2):

```
service routing protocols model multi-agent
!
router bgp 65000
  router-id 10.255.0.1
  no bgp default ipv4-unicast
  neighbor EVPN-PEERS peer group
  neighbor EVPN-PEERS remote-as 65000
  neighbor EVPN-PEERS update-source Loopback0
  neighbor EVPN-PEERS route-reflector-client
  neighbor EVPN-PEERS send-community extended
  neighbor 10.255.0.11 peer group EVPN-PEERS
  neighbor 10.255.0.12 peer group EVPN-PEERS
!
address-family evpn
  neighbor EVPN-PEERS activate
```

Leaves — clients, peering to both spines. On leaf1 (leaf2: router-id 10.255.0.12):

```
service routing protocols model multi-agent
!
router bgp 65000
  router-id 10.255.0.11
  no bgp default ipv4-unicast
  neighbor EVPN-PEERS peer group
  neighbor EVPN-PEERS remote-as 65000
  neighbor EVPN-PEERS update-source Loopback0
  neighbor EVPN-PEERS send-community extended
  neighbor 10.255.0.1 peer group EVPN-PEERS
  neighbor 10.255.0.2 peer group EVPN-PEERS
!
address-family evpn
  neighbor EVPN-PEERS activate
```

Reading notes, because copying without reading is banned here: `multi-agent` is the EOS routing process model EVPN requires (on real hardware it needs a restart; cEOS just takes it — if EVPN routes mysteriously don't appear later, `write mem` and redeploy the lab, the saved config boots with it active). `update-source Loopback0` is why module 03 obsessed over loopback reachability — these sessions ride the underlay between /32s. `no bgp default ipv4-unicast` keeps this BGP instance purely for EVPN; OSPF still owns the underlay. One AS, iBGP, spines as RRs — the classic simple EVPN design.

Checkpoint before continuing — on a spine:

```
show bgp evpn summary
```

Both leaves `Estab`. Not established? It's underlay reachability 90% of the time: `ping 10.255.0.11 source 10.255.0.1`, and check OSPF is still FULL. Fix the underlay first, always.

4. VXLAN data plane (leaves only)

On both leaves, identically:

```
interface Vxlan1
  vxlan source-interface Loopback0
  vxlan udp-port 4789
  vxlan vlan 100 vni 10100
!
router bgp 65000
  vlan 100
  rd auto
```

```
route-target both 10100:100
redistribute learned
```

That maps VLAN 100 ↔ VNI 10100, anchors the tunnel on Loopback0, and tells BGP to advertise MACs learned in VLAN 100 into EVPN with a route-target both leaves import.

5. The finish line

From leaf1:

```
ping 172.16.100.2
```

If that answers, an L2 frame just got VXLAN-encapsulated on leaf1, routed across your OSPF fabric as UDP, and decapsulated on leaf2. Prove to yourself it's real, not magic:

```
show bgp evpn route-type mac-ip      ! leaf2's MAC, learned via BGP, next-hop = leaf2's VTEP
show vxlan address-table              ! remote MACs and which VTEP they live behind
show interfaces Vxlan1                ! VNI up, flood list = the other VTEP
traceroute 172.16.100.2               ! one hop! the fabric is invisible to the overlay
```

That one-hop traceroute is the whole concept in a single line: the overlay thinks it's a cable; the underlay did all the work.

Then break it the instructive way: shutdown leaf1's Ethernet1 and ping again — still works, the *tunnel* rerouted over spine2 because the underlay reconverged and the overlay never noticed. That indirection is why data centers are built this way.

6. Capstone: automate it

You just typed near-identical config on four nodes again. You know what happens now. Build, using module 04's exact model:

1. templates/spineleaf_evpn.j2 — the config above with holes. Design the template to handle both roles (`{% if host.data.role == "spine" %}` for the RR block) or make two templates; both are defensible, pick one and be able to say why.
2. An EVPN data dict — router-ids, peer lists, and per-leaf VLAN/VNI/SVI values.
3. 06_spineleaf_evpn.py — copy 05_spineleaf_underlay.py, point it at your template and data.

Definition of done — the same bar every module 04 script met:

Fresh fabric (destroy, delete `clab-spineleaf/`, deploy) reaches “SVI pings SVI” by running exactly two scripts: 05 then 06

Both scripts, run a second time: `changed=False` everywhere

Stretch: extend module 04's `verify_fabric.py` to also assert EVPN sessions are `Estab` and the `mac-ip` route for the far SVI exists

That first box is worth pausing on when you tick it: a two-command, reproducible, self-verifying EVPN fabric. That's not a lab trick — that's how modern data center networks are actually operated, and you just built one end to end.

What you learned

Overlay/underlay separation and why each layer stays ignorant of the other; VTEPs, VNIs, and EVPN as a BGP address family; spines as route reflectors; the debugging hierarchy (underlay first, control plane second, data plane last); and — via the capstone — that the automation model from module 04 didn't change at all when the config got 10x more sophisticated. Template, data, idempotent push. It never changes.

Next (optional): [Module 06 — Secrets](#)